

## TD n° 3 : Grammaires algébriques

*Objectif : concevoir et manipuler des grammaires algébriques.*

## Exercice 1 — Ambiguïté

Soit la grammaire  $G = (\Sigma_{NT}, \Sigma_T, P, S)$ , où  $\Sigma_{NT} = \{S, A\}$ ,  $\Sigma_T = \{a, b\}$ , et

$$P = \{ \begin{array}{l} S \rightarrow A A, \\ A \rightarrow A A A, \\ A \rightarrow b A, \\ A \rightarrow A b, \\ A \rightarrow a \end{array} \}.$$

- Donner une dérivation gauche (chercher ce que c'est) pour le mot "aba" ; dessiner l'arbre de dérivation correspondant ;
- Donner une dérivation droite (chercher ce que c'est) pour le mot "aba" qui produise le même arbre de dérivation ;
- Donner plusieurs arbres de dérivation pour le mot "ababaa".

## Réflexions :

- a. On démarre obligatoirement du symbole non-terminal axiome, soit  $S$ . La dérivation commence donc par

$$S \rightarrow A A \rightarrow \dots$$

- b. On observe qu'aucune règle n'engendre le vide,  $\varepsilon$ . Le nombre de lettres terminales engendrées est donc supérieur ou égal au nombre de lettres non terminales engendrées à un point donné de la dérivation. En conséquence utiliser la seconde règle produirait la séquence 'A A A A' qui ne pourra pas se dériver en 'aba'.
- c. On observe aussi que la troisième règle engendre un 'b' à gauche alors qu'un 'a' est attendu.
- d. En tâtonnant un peu on trouve la dérivation gauche suivante :

$$S \rightarrow \underline{A} A \rightarrow \underline{A} b A \rightarrow a b \underline{A} \rightarrow a b a$$

et en tâtonnant un peu plus, on trouve une autre dérivation gauche !

$$S \rightarrow \underline{A} A \rightarrow a \underline{A} \rightarrow a b \underline{A} \rightarrow a b a$$

et une dérivation droite

$$S \rightarrow A \underline{A} \rightarrow A b \underline{A} \rightarrow \underline{A} b a \rightarrow a b a$$

- e. Et enfin pour le mot "ababaa"

$$\begin{aligned} S &\rightarrow \underline{A} A \rightarrow A A \underline{A} \rightarrow \underline{A} A A A \\ &\rightarrow \underline{A} b A A A \rightarrow a b \underline{A} A A \rightarrow a b \underline{A} b A A \\ &\rightarrow a b a b \underline{A} A \rightarrow a b a b a \underline{A} \rightarrow a b a b a a \end{aligned}$$

ou

$$\begin{aligned} S &\rightarrow \underline{A} A \rightarrow A A A A \\ &\rightarrow \underline{A} b A A A \rightarrow a b \underline{A} A A \rightarrow a b \underline{A} b A A \\ &\rightarrow a b a b \underline{A} A \rightarrow a b a b a \underline{A} \rightarrow a b a b a a \end{aligned}$$

et de nombreuses autres dérivations possibles.

### Questions :

- a. Tracer les arbres de dérivation.
- b. Constaté que des dérivations différentes peuvent produire des arbres identiques. Évidemment, le contraire n'est pas possible !

### Remarques :

- a. Il est important de lire les règles de façon dynamique : remplacer une occurrence d'une partie gauche de règle par la partie droite correspondante. Dans le cas des grammaires algébriques, la partie gauche est nécessairement une lettre non terminale.
- b. Une dérivation est alors comme une succession de « coups » (échec ou dames). Il faut alors penser une dérivation comme un chemin entre une situation de départ et une situation finale désirée.
- c. Une stratégie est alors une démarche systématique pour trouver des dérivations.
- d. Une stratégie peut être efficace (trouver rapidement une dérivation, ou trouver une dérivation courte), correcte (ne proposer que des dérivations correctes ; c'est le moins qu'on puisse demander), ou complète (trouver une dérivation à chaque fois qu'il en existe).
- e. De nombreuses stratégies intéressantes sont efficaces, correctes, mais incomplètes. L'incomplétude est le prix de l'efficacité !
- f. La stratégie de dérivation à gauche (symétriquement à droite) est correcte ; si il existe des dérivations vers un mot donné au moins une est une dérivation à gauche. Pareil pour les dérivations à droite. On ne se prive donc de rien en se restreignant à des dérivations gauches (ou droites).

### Exercice 2 — Conception de grammaires

Pour chacun des traits de langages suivants, en donner une description dans le langage des grammaires algébriques. On pourra utiliser la variante étendue des grammaires algébriques.

- Expression à la LISP : un opérateur est toujours noté en position préfixe, suivi de ses opérandes séparés par des espaces, le tout entouré de parenthèse. Un opérateur peut avoir un nombre varié d'opérandes : ex.

(plus (mul a (exp X 2)) (mul b X) C) .

- Expression arithmétique à la Java : un opérateur est noté en position infixe quand c'est l'usage, avec des parenthèses si besoin pour forcer les priorités. Ex.

$a * x^{**}2 + (-b) * X + c$  .

- Expression arithmétique comme les mathématiciens : l'opérateur de multiplication peut être omis. Ex.

$a x^{**}2 + (-b) X + c$  .

- Liste de paramètres effectifs comme en Java : liste éventuellement vide de paramètres effectifs séparés par des virgules. Ex.

L3, "ISTIC", 12 .

- Liste de paramètres effectifs nommés comme en Python : liste éventuellement vide d'affectations de paramètres effectifs à des paramètres formels. Ex.

ecole=ISTIC, num\_étudiant=12, formation=L3 .

- Instruction conditionnelle à la Java : mot-clé 'if', suivi de l'expression conditionnelle entre parenthèses, suivi de la branche 'then' obligatoire, puis de la branche 'else' optionnelle. La branche 'then' n'est introduite par aucun mot-clé. La branche 'else' est introduite par le mot-clé 'else'. Si les branches 'then' ou 'else' sont formées de plusieurs instructions, elles sont délimitées par des accolades. Les accolades sont admises, mais pas obligatoires, pour les branches formées d'une unique instruction. Ex.

```
if (X > 0) X = X - 1
else {
    X = X + 1 ;
    if (X != -12) {
        X = 12
    }
}
```

- Expression conditionnelle à la Java : une expression conditionnelle suivie d'un '?' suivie de la valeur de l'expression si la condition est vraie, suivie de ':', suivie de la valeur si la condition est fausse, le tout entre parenthèses. Ex.

( X < 0 ? -1 : (X == 0 ? 0 : 1) ) .

### Réflexions :

- Chaque expression informelle d'un trait de langage « raconte une histoire ». Il faut trouver un moyen de raconter la même histoire avec une grammaire algébrique.
- Expression à la LISP :

```
Expr → '(' Op ( ' ' Arg)* ')' | Primary
Op → Expr
Arg → Expr
Primary → Id | Num
Id → [a – z A – Z] [a – z A – Z 0 – 9]*
Num → [0 – 9] [0 – 9]*
```

Attention aux parenthèses du langage, et à celle du méta-langage (1<sup>ère</sup> règle).

La question ne disait rien des identificateurs et des nombres, mais l'exemple fourni fait penser que...

La question ne dit rien de ce que sont Op et Arg, mais l'exemple et la culture générale aident à s'en faire une idée.

- Expressions arithmétiques à la Java :

```
Expr → Sum | Diff | Opp | Prod | Div | Pow | Parens
Sum → Expr '+' Expr
Diff → Expr '-' Expr
Opp → Expr '-' Expr
...
est tentant, mais très ambigu (revoir la définition de l'ambiguïté). Il faut préférer
```

```
Expr → SumDiff
SumDiff → ProdDiv | SumDiff ('+' | '-') ProdDiv
ProdDiv → Unary | ProdDiv ('*' | '/') Unary
```

Unary  $\rightarrow$  ('-')? Pow  
 Pow  $\rightarrow$  Nullary | Nullary '\*' '\*' Pow  
 Nullary  $\rightarrow$  Number | Parens  
 Number  $\rightarrow$  ...  
 Parens  $\rightarrow$  '(' Expr ')'

- d. Expressions arithmétiques comme les mathématiciens, (c-à-d. en n'écrivant pas les signes de multiplication). On supposera aussi que les puissances,  $x^y$ , sont écrites  $x^y$  :

Expr  $\rightarrow$  SumDiff  
 SumDiff  $\rightarrow$  ProdDiv | SumDiff ('+' | '-') ProdDiv  
 ProdDiv  $\rightarrow$  Unary | ProdDiv (' ' | '/') Unary  
 Unary  $\rightarrow$  ('-')? Pow  
 Pow  $\rightarrow$  Nullary | Nullary '^' Pow  
 Nullary  $\rightarrow$  Number | Var | Parens  
 Number  $\rightarrow$  ...  
 Var  $\rightarrow$  [a – z] [0 – 9]\*  
 Parens  $\rightarrow$  '(' Expr ')'

- e. Paramètres effectifs à la Java :

Params  $\rightarrow$  Expr (',' Expr)\*

Il est inutile de développer le non-terminal Expr. On suppose que les langages d'expressions pour les différents types de Java sont définis.

- f. Paramètres effectifs nommés à la Python :

Params  $\rightarrow$  VarId '=' Expr (',' VarId '=' Expr)\*

- g. Instruction conditionnelle à la Java :

Instr  $\rightarrow$  InstrWhile | InstrCond | ...  
 InstrSeq  $\rightarrow$  Instr ( ';' Instr)\*  
 InstrCond  $\rightarrow$  'if' ExprCond Block ('else' Block)?  
 Block  $\rightarrow$  Instr  
 Block  $\rightarrow$  '{' InstrSeq '}'

On peut discuter du fait qu'un bloc '{ }' soit admis ou non.

- h. Expression conditionnelle à la Java :

Expr  $\rightarrow$  '(' ExprCond '?' Expr ':' Expr ')'

Il n'est pas nécessaire de développer plus car ExprCond et Expr sont supposés être des symboles non-terminaux existants (ça existe certainement).

### Questions :

- a. Pourquoi

Expr  $\rightarrow$  '(' Op (' ' Arg)\* ')'

et pas

Expr  $\rightarrow$  '(' Expr (' ' Expr)\* ')'

- b. Pourquoi

Pow  $\rightarrow$  Nullary | Nullary '\*' '\*' Pow

et pas

Pow  $\rightarrow$  Nullary | Pow '\*' '\*' Nullary

### Remarques :

- Ne pas hésiter à utiliser une notation de grammaire algébrique étendue (revoir la définition), surtout pour une grammaire papier. Pour sa mise en œuvre avec outil faire ce que l'outil permet, mais si on a le choix disposer de la notation étendue est un bon critère de choix.
- En première approche, une grammaire engendre les mots d'un langage. Mais en seconde approche, elle engendre les arbres de dérivation dont les mots sont la concaténation des feuilles. C'est la seconde approche qui est la plus importante, car c'est là qu'on voit que la syntaxe préfigure la sémantique.
- La curiosité pour les langages de programmation est un impératif pour les informaticiens. Dans « *The Pragmatic Programmer* », Andy Hunt et Dave Thomas (créateurs entre autres de JUnit) disent même qu'un informaticien responsable doit se donner pour objectif d'apprendre un langage de programmation tous les ans.
- Comprendre un langage de programmation n'est pas que savoir où on met les ponctuations et les parenthèses, c'est surtout en connaître la structure, et ça c'est ce qu'offrent les grammaires.

### Exercice 3 — Reformulation de grammaire pour prendre en compte la portée des opérateurs

Soit la grammaire G

$$\begin{aligned} E &\rightarrow a \mid b \mid c \\ &\mid E '+' E \\ &\mid E 'x' E \\ &\mid '(' E ')'. \end{aligned}$$

- Construire un arbre de dérivation de l'expression  $a + b \times c$ . Que remarque-t-on ?
- Réécrire la grammaire G en  $G_{\text{new}}$  de façon à résoudre le problème observé.
- Construire l'arbre de dérivation de  $a + b \times c$  avec  $G_{\text{new}}$ .

### Réflexions :

- On trouve facilement une dérivation

$$\begin{aligned} E &\rightarrow \underline{E} '+' E \rightarrow a '+' \underline{E} \rightarrow a '+' \underline{E} 'x' E \\ &\rightarrow a '+' b 'x' E \rightarrow a '+' b 'x' c \end{aligned}$$

et donc un arbre de dérivation.

- Mais, on trouve facilement une autre dérivation

$$\begin{aligned} E &\rightarrow \underline{E} 'x' E \rightarrow \underline{E} '+' E 'x' E \rightarrow a '+' \underline{E} 'x' E \\ &\rightarrow a '+' b 'x' \underline{E} \rightarrow a '+' b 'x' c \end{aligned}$$

qui engendre le même mot mais un arbre de dérivation différent. Cette grammaire est donc ambiguë.

- L'ambiguïté vient de ce que les deux alternatives traitent l'opérande de gauche comme celui de droite. Il faut les différencier.
- Nous avons tous appris à la petite école la façon de lire des sommes de plus de deux opérandes :  $1 + 2 + 3 + \dots 100$ . Sans y penser vraiment,

nous lisons cela comme  $1 + 2$ , auquel on ajoute 3, puis 4, etc. jusqu'à 100. On considère donc qu'un '+' a pour opérande gauche une somme et pour opérande droite une expression de moindre portée. Essayons de raconter cela avec la grammaire  $G_{\text{new}}$  :

$$\begin{aligned} E &\rightarrow S \\ S &\rightarrow S '+' P \mid P \\ P &\rightarrow P 'x' F \mid F \\ F &\rightarrow a \mid b \mid c \mid '(' E ')' . \end{aligned}$$

$G_{\text{new}}$  raconte l'histoire suivante. Les F (ou facteurs) sont des expressions élémentaires, sans opérateur, ou des expressions quelconques mises entre parenthèses. Les P (ou produits) sont des produits ou des facteurs. Si ce sont des produits, l'opérande de gauche peut aussi être un produit, mais pas l'opérande de droite. Les S (ou sommes) sont des sommes ou des produits. Si ce sont des sommes, l'opérande de gauche peut aussi être une somme, mais pas l'opérande de droite. La règle  $E \rightarrow S$  n'est pas strictement nécessaire ; elle rend juste l'histoire plus facile à raconter.

- e. On peut dériver  $a + b \times c$  de la façon suivante :

$$\begin{aligned} E &\rightarrow S \\ &\rightarrow S '+' P \\ &\rightarrow P '+' P \\ &\rightarrow E '+' P \\ &\rightarrow a '+' P \\ &\rightarrow a '+' P 'x' F \\ &\rightarrow a '+' E 'x' F \\ &\rightarrow a '+' b 'x' E \\ &\rightarrow a '+' b 'x' c \end{aligned}$$

Se rappeler que se limiter aux dérivations à gauche ne prive de rien.

### Questions :

- Rappeler la définition de l'ambiguïté d'une grammaire.
- Rappeler les trois façons de corriger une grammaire ambiguë.
- Que peut-on reprocher à la troisième façon ?
- Rappeler les définitions de portée et priorité.
- Essayer de dériver  $(a + b) \times c$  avec la grammaire  $G_{\text{new}}$ .

### Remarques :

- Être ambigu est vraiment grave. On ne doit pas tolérer qu'une grammaire soit ambiguë.
- Être ambigu n'est pas une propriété d'un langage (sauf cas extrême où un langage ne pourrait être engendré que par des grammaires ambiguës). C'est fondamentalement une propriété d'une grammaire.
- On trouve souvent plus facilement des grammaires non ambiguës pour des langages très parenthésés, comme Lisp ou HTML.

#### Exercice 4 — Reformulation de grammaire pour prendre en compte la stratégie LL(1)

Soit les grammaires  $G =$

$$\begin{aligned} S &\rightarrow a X \\ X &\rightarrow b \\ X &\rightarrow c \end{aligned}$$

et  $H =$

$$\begin{aligned} S &\rightarrow a B \\ S &\rightarrow a C \\ B &\rightarrow b \\ C &\rightarrow c \end{aligned}$$

avec  $\Sigma_T = \{ a, b, c \}$ .

- Comparer les langages  $\mathcal{L}(G)$  et  $\mathcal{L}(H)$
- La grammaire  $G$  est-elle LL(1) ? La grammaire  $H$  est-elle LL(1) ?

##### Réflexions :

- a. La grammaire  $G$  est certainement LL(1) car à chaque paire <non-terminal, terminal> correspond une unique règle.
- b. La grammaire  $H$  n'est pas LL(1), car à la paire  $(S, a)$  correspond deux règles entre lesquelles rien ne permet de choisir.

##### Questions :

- a. Rappeler la définition de LL(1).

##### Remarques :

- a. Être LL(1) est une propriété d'une grammaire ou d'un langage qui peut être engendré par une grammaire LL(1).
- b. Ne pas être LL(1) n'est pas grave en soit. Ce qui est grave c'est de ne disposer que d'une grammaire qui n'est pas LL(1) alors qu'on a l'intention d'utiliser une stratégie d'analyse qui est LL(1).
- c. Dans cette situation, on peut opter pour une autre grammaire qui serait LL(1), ou on peut décider de changer de stratégie d'analyse.

#### Exercice 5 — Reformulation de grammaire pour prendre en compte la récursivité gauche

Soit la grammaire  $G$  de l'exercice 3.

- Cette grammaire est-elle ambiguë ? Est-elle LL(1) ? Est-elle récursive à gauche ?
- La grammaire  $G_{\text{new}}$  est-elle ambiguë ? Est-elle LL(1) ? Est-elle récursive à gauche ?
- La réécrire en syntaxe EBNF.

### Réflexions :

- On a vu que la grammaire  $G$  de l'exercice 3 est ambiguë.
- Il est facile de montrer que  $G$  n'est pas  $LL(1)$  et qu'elle est récursive à gauche, mais on ne le montrera que pour  $G_{new}$ .
- Il se trouve que la grammaire  $G_{new}$  proposée pour corriger cela n'est pas ambiguë (on a tout fait pour), mais qu'elle n'est pas  $LL(1)$ , et qu'elle est récursive à gauche !
- $G_{new}$  est récursive à gauche ; ça se voit. Ex.  $S \rightarrow S '+' P$  .
- Déterminer que  $G_{new}$  n'est pas  $LL(1)$  est à peine plus délicat, car on observe facilement qu'à la paire  $\langle E, a \rangle$  correspondent plusieurs règles :  

$$S \rightarrow S '+' P \text{ et } S \rightarrow P$$
- La notation de grammaire étendue (EBNF) offre un bon moyen de supprimer les récursivités à gauche :  

$$S \rightarrow P ('+' P)^*$$

$$P \rightarrow F ('x' F)^*$$

$$F \rightarrow a \mid b \mid c \mid ('E')$$

### Questions :

- Rappeler l'autre façon d'éliminer les récursivités à gauche.
- Appliquer cette autre façon à  $G_{new}$ .
- Qu'en pensez-vous ?
- Qui est Sheila Greibach ?
- Montrer que  $G$  n'est pas  $LL(1)$ .

### Remarques :

- EBNF stricto-sensu est plus que le concept de grammaire algébrique étendue. C'est aussi une écriture particulière de ces grammaires. Mais ce qui compte vraiment, c'est le concept.
- La notation étendue offre un confort d'écriture très appréciable. C'est vraiment un critère déterminant pour le choix d'un outil d'analyse par grammaire algébrique.
- Bien réaliser que pour l'ambiguïté la faute est à rechercher dans la grammaire, mais que pour le fait de ne pas être  $LL(1)$  ou d'être récursive la faute est à rechercher dans l'adéquation entre la forme d'une grammaire et un type de stratégie d'analyse.
- D'une manière générale, il est beaucoup plus facile de montrer qu'une grammaire n'est pas  $LL(1)$  que de montrer qu'elle l'est. En effet, dans le premier cas, un contre-exemple suffit, c'est-à-dire trouver une paire  $\langle \text{terminal}, \text{non-terminal} \rangle$  pour qui plusieurs règles peuvent être activées. Dans le second cas, il faut montrer qu'il n'existe aucun contre-exemple, et il faut alors utiliser l'algorithme vu en cours.